GraphQL for System Design

resolvers, N+1, DataLoader and query limits

Watch the 4-min video

In one line: GraphQL lets each client ask a typed schema for exactly the data a screen needs in one request; the server pays for that flexibility with N+1 risk (fix: DataLoader), expensive-query risk (fix: depth and cost limits) and harder caching.

1 The problem: over-fetching and under-fetching

A mobile product page needs the product, its price, three reviews, the seller and related items. With a typical REST API that is **five round trips** over a slow cellular network. That is **under-fetching**: one call does not return enough. And each response carries fields the screen never shows: **over-fetching**.

GraphQL lets the client ask for exactly what it needs, in one request.

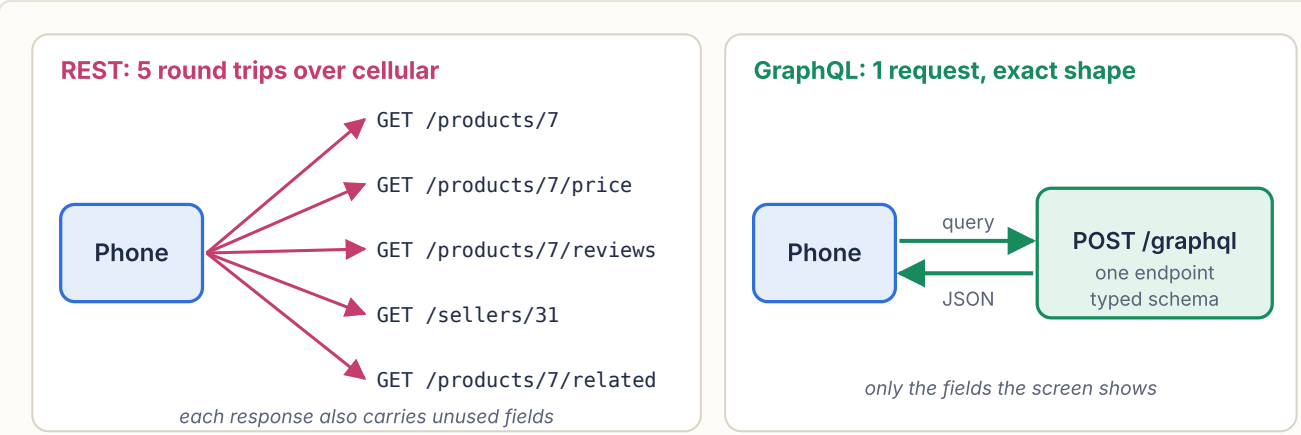


Figure 1. At ~100 ms per cellular round trip, five calls made one after another cost about half a second before any rendering; one GraphQL request costs one round trip.

2 Schema, operations and one endpoint

GraphQL starts with a **typed schema**: types such as `Product`, `Review` and `Seller`, with their fields and relationships.

```
type Product { id: ID!  name: String!  price: Float!
               reviews(first: Int): [Review!]!  seller: Seller!  related: [Product!]! }
type Review  { rating: Int!  text: String }
type Seller  { name: String! }
type Query   { product(id: ID!): Product }
```

- **Queries** read data.
- **Mutations** change it (e.g. `addReview`).
- **Subscriptions** push updates to the client, usually over WebSockets.

Everything goes through a **single endpoint**, typically `POST /graphql`. The schema is the contract, and tools validate every query against it.

3 The product page as one query

Query (client writes it)

```
query ProductPage {
  product(id: "7") {
    name
    price
    reviews(first: 3) {
      rating
      text
    }
    seller { name }
    related { name }
  }
}
```

Response (same shape)

```
{ "data": {
  "product": {
    "name": "Trail shoe",
    "price": 89.0,
    "reviews": [
      {"rating":5,"text":".."},
      ... 2 more
    ],
    "seller": {"name":"Acme"},
    "related": [{"name":..}]
  }
} }
```

Figure 2. The client names the product by ID and lists the fields it wants: name, price, the first three reviews with rating and text, the seller's name, related product names. The JSON has exactly that shape.

No over-fetching of unused fields, and no under-fetching that forces extra calls.

4 Resolvers and the N+1 problem

On the server, **every field has a resolver**: a function that fetches its data. That is flexible, but it hides a trap. Fetch a list of 50 products, and the `reviews` resolver runs once per product: **1 query for the list + 50 for the reviews = 51 database queries** for a single request.

The fix is **DataLoader**. Instead of querying immediately, each resolver asks a loader for a key. The loader collects all keys requested in the same tick (one turn of the event loop) and runs **one batched query**: reviews where `product_id IN (…)`. It also **caches within the request**, so the same seller requested ten times is loaded once.

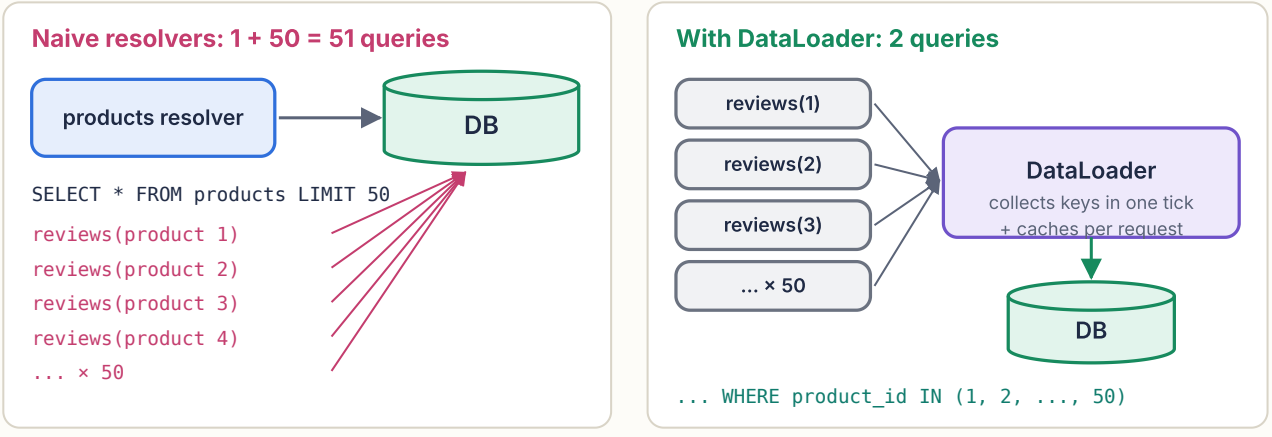


Figure 3. Left: one query per parent row. Right: the loader batches all 50 keys into one IN query. 51 queries become 2.

WORKED EXAMPLE

A list page shows 50 products, each with reviews and its seller. Naive: 1 (products) + 50 (reviews) + 50 (sellers) = **101 queries**. At 2 ms each that is about 200 ms of database time, and the database works 50× harder than it needs to.

With DataLoader: 1 (products) + 1 (reviews `IN` 50 ids) + 1 (sellers `IN` the distinct seller ids, say 12) = **3 queries**, about 6–10 ms.

5 Protecting the server from expensive queries

Clients write the queries, so a careless or malicious one can nest deeply: products → reviews → each reviewer's orders → each order's products → and so on. Each level multiplies the work. Defend in layers:

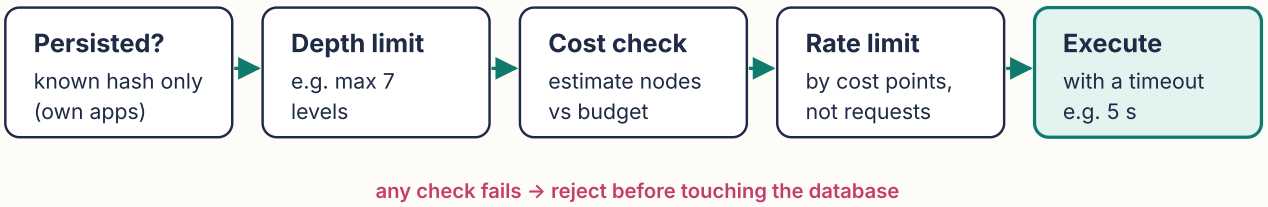


Figure 4. Checks run on the parsed query, before any resolver runs.

- **Depth limit**: reject queries nested beyond a fixed depth.
- **Query cost analysis**: estimate how many nodes a query can return, using list sizes. `products(first: 50) { reviews(first: 10) { author { orders(first: 20) } } }` is 50 × 10 × 20 = **10,000** nodes; reject anything above the budget.
- **Timeouts** on execution, and **rate limits based on cost**, not request count: one huge query should use as much budget as many small ones.
- **Persisted queries** for your own apps: at build time each query is registered, and clients send only its hash. The server runs only known queries.

6 Caching, and when to choose GraphQL

Caching is harder than with REST. Everything is a POST to one URL, so CDNs and browser HTTP caches cannot help by default. Two things fill the gap: **persisted queries sent over GET** (the hash and variables go in the URL, so a CDN can cache them), and **normalized client caches** such as Apollo's, which store each object once by type and ID.

GraphQL fits well	REST is usually simpler
Many different clients (especially mobile) needing flexible, aggregated views of related data	Simple CRUD services
A backend-for-frontend that aggregates several services per screen	Public, cache-friendly APIs served through a CDN
Fast-changing UIs that would otherwise need new endpoints each release	File uploads and downloads

COMMON MISTAKES

- Shipping resolvers without DataLoader, then wondering why one request makes hundreds of queries.
- Exposing a public GraphQL endpoint with no depth or cost limits.
- Rate limiting by request count: one request can be 10,000× more expensive than another.
- Expecting the CDN to cache `POST /graphql` responses.
- Choosing GraphQL for a simple CRUD service with one client, and taking on its complexity for nothing.

7 Key terms

Schema: typed contract of types and fields

Resolver: function that fetches one field

Over-fetching: fields you receive but don't use

Query / mutation: read / change data

N+1 problem: 1 list query + 1 query per item

Query cost: estimated nodes a query may touch

Subscription: server push, usually over WebSockets

DataLoader: batches and caches keys per request

Persisted query: pre-registered query sent by hash

8 Interview questions

Q1. When is GraphQL better than REST? BASIC

When diverse clients, like mobile apps and a backend-for-frontend, need flexible, aggregated data in one round trip without over- or under-fetching. Five REST calls at ~100 ms on cellular become one request.

Q2. What is the N+1 problem in GraphQL? BASIC

A list resolver runs one query, then a child resolver runs once per item. 50 products with reviews = 1 + 50 = 51 queries. It comes from resolving each field independently.

Q3. How does DataLoader fix N+1? BASIC

Resolvers ask the loader for keys instead of querying. It gathers all keys requested in the same tick and runs one batched query (WHERE product_id IN (...)), and caches results for the rest of that request. 51 queries become 2.

Q4. How do you prevent expensive GraphQL queries? SCENARIO

Depth limits; cost analysis that multiplies list sizes (50 × 10 × 20 = 10,000 nodes) and rejects queries over budget; execution timeouts; cost-based rate limiting; persisted queries for first-party apps; and DataLoader so legitimate queries stay cheap.

Q5. Your product page API is slow on mobile. Would you add GraphQL? SCENARIO

If the page needs data from several resources and each call is a cellular round trip, a GraphQL BFF that aggregates them on the server side helps: the slow hops move to the fast data-centre network. If the slowness is one slow database query, GraphQL will not fix it.

Q6. Why is caching GraphQL harder than caching REST, and what helps? TRICKY

All requests are POST to one URL, so HTTP caches and CDNs cannot key on the URL. Use persisted queries over GET so the URL identifies the query, and a normalized client cache (e.g. Apollo) keyed by type + ID.

Q7. A single client sends 10 requests per minute but your database is overloaded. How can that be? TRICKY

Each request can be a deeply nested query touching thousands of rows. Rate limits by request count miss that. Score each query by cost and limit points per client per minute, and set depth and node caps.

Next: SD-C-003 · gRPC & Protocol Buffers backendroadmap.com